Compiler Lab — final_to_study Guide

Oct 23, 2026

This guide explains every line of the code in the `final_to_study` folder, in the order you should study it.

1. How to run it, and which file answers which question

The compiler is four small programs, one per exam module, joined by a script. Each program can be shown on its own, because each module carries its own marks.

Question	File	What it does
(a) Lex specification	`lexer.l`	splits the input into tokens and prints each one
(b) Yacc syntax check	`parser.y`	checks the grammar; prints `Syntax is valid` or `Syntax error at line N`
(c) SDT for three-address code	`parser.y`	the `{ }` actions print the three-address code (TAC)
(d) Optimization	`opt.c`	constant folding, algebraic simplification, strength reduction, code motion
(e) Target code	`gen.c`	turns the optimized TAC into assembly
(f) Integration	`run.sh`	builds everything and runs (a) to (e) in order

Run everything at once (uses `input.txt`, the exam sample):

```
sh run.sh
sh run.sh tests/for.txt
```

Run one module at a time:

```
bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o icg
gcc opt.c -o opt
gcc gen.c -o gen

./icg < input.txt > tac.txt
./opt < tac.txt > opt.txt
./gen < opt.txt
```

- `bison -d parser.y` makes `parser.tab.c` and `parser.tab.h`. Run it before `flex`, because `lexer.l` includes `parser.tab.h`.
- `./icg < input.txt > tac.txt` shows the tokens and syntax message on screen and saves the TAC in `tac.txt`.
- If the lab only has `lex` and `yacc`, use `yacc -d -b parser parser.y` and `lex lexer.l`. Everything else is the same.

The `tests` folder holds inputs for `for`, `if`, `if-else`, expressions, nesting and a syntax error.

2. The big picture: one statement through all five stages

A compiler works in stages, and each stage's output is the next stage's input. Here the stages talk through text files:

```
input.txt --[icg: lexer.l + parser.y]--> tac.txt --[opt]--> opt.txt --[gen]--
> assembly
               (a) tokens, (b) syntax,              (d)               (e)
               (c) three-address code
```

- **lexer.l** cuts the text into tokens and hands them to the parser one at a time.
- **parser.y** checks the tokens follow the grammar. Each time a rule matches, its action prints TAC.
- **opt.c** reads the TAC, improves it, and prints the better version.
- **gen.c** reads the optimized TAC and prints assembly.

Follow `y = b * 4;` through every stage. All of this is real output.

(a) Tokens, printed by the lexer:

```
y          identifier
=          assignment operator
b          identifier
*          arithmetic operator
4          constant
;          special symbol
```

(b) Syntax. The tokens match the rule `STMT : ID '=' E ';'`, so the parser prints `Syntax is valid`.

(c) Three-address code. Each line has at most one operator; `t1` is a temporary:

```
t1 = b * 4
y = t1
```

(d) Optimized. Multiplying by 4 is replaced by a cheaper shift left by 2 (strength reduction):

```
t1 = b << 2
y = t1
```

(e) Assembly. Each TAC line becomes load, operate, store using register `R0`:

```
MOV R0, b
SHL R0, 2
MOV t1, R0
MOV R0, t1
MOV y, R0
```

The rest of this guide looks at each arrow in detail.

3. lexer.l line by line (question a)

The lexer turns characters into **tokens**: the words of the language, such as a keyword, a name, a number or an operator. The parser calls it again and again, and each call returns the next token.

A lex file has three parts separated by `%%`:

1. C code between `%{` and `%}`: includes and helper functions.
2. Rules: `pattern { action }`. When the input matches a pattern, its action runs.
3. Plain C after the second `%%`.

```
%{
#include <stdio.h>
#include <string.h>
#include "parser.tab.h"

void tok(char *kind) { fprintf(stderr, "%-10s %s\n", yytext, kind); }
%}

%option yylineno

%%
"if"                    { tok("keyword"); return IF; }
"else"                  { tok("keyword"); return ELSE; }
"while"                 { tok("keyword"); return WHILE; }
"for"                   { tok("keyword"); return FOR; }

"<"|">"|"<="|">="|"=="|"!="  { tok("relational operator"); yylval.s =
strdup(yytext); return RELOP; }

[a-zA-Z_][a-zA-Z0-9_]*  { tok("identifier"); yylval.s = strdup(yytext);
return ID; }
[0-9]+                  { tok("constant"); yylval.s = strdup(yytext); return
NUM; }

[-+*/]                  { tok("arithmetic operator"); return yytext[0]; }
"="                     { tok("assignment operator"); return '='; }
[(){};]                 { tok("special symbol"); return yytext[0]; }

[ \t\r\n]+              ;
.                       { tok("invalid character"); }
%%

int yywrap() { return 1; }
```

Line	Meaning
`#include <stdio.h>`	gives `fprintf`
`#include <string.h>`	gives `strdup` (makes a copy of a string)
`#include "parser.tab.h"`	made by `bison -d`; it defines the token names `IF`, `ID`, `NUM`, `RELOP` … and the variable `yylval`

Line	Meaning
`void tok(char *kind)`	helper that prints the matched text and its kind, e.g. `while keyword`
`yytext`	the exact text the current pattern matched
`fprintf(stderr, ...)`	prints to the screen even when normal output goes to a file, so tokens never mix into `tac.txt`
`%-10s`	prints the text left-aligned in 10 columns, so the list lines up
`%option yylineno`	lex counts lines in `yylineno`; the parser uses it in error messages
`"if" { ... return IF; }`	the keyword `if`: print it, then tell the parser "this token is IF"
`"<"|">"|...`	any of the six comparison operators (`|` means "or")
`yylval.s = strdup(yytext)`	a token can carry a value; this stores a copy of the text (`<`, the name, or the digits) for the parser, which reads it as `$1`, `$2` ...
`[a-zA-Z_][a-zA-Z0-9_]*`	an identifier: a letter or `_`, then letters, digits or `_`
`[0-9]+`	a number: one or more digits
`[-+*/]`	one arithmetic operator; `-` is first so it is not read as a range
`return yytext[0];`	for a one-character symbol the token is the character itself, so the grammar writes `'+'`, `';'`
`[ \t\r\n]+ ;`	spaces, tabs and newlines: empty action, so they are skipped
`.`	any other single character: reported as invalid
`yywrap()` returns 1	called at end of input; 1 means "stop, no more files"

Two rules decide which pattern wins:

- **Longest match.** `<=` is one token, not `<` then `=`.

- **First rule on a tie.** `while` matches both the keyword rule and the identifier rule with 5 letters. The keyword rule is listed first, so it wins. That is why keywords come before identifiers. A longer name like `whiles` is still an identifier.

Check yourself: what does the lexer print for `i = i + 1;`? Answer: `i` identifier, `=` assignment operator, `i` identifier, `+` arithmetic operator, `1` constant, `;` special symbol.

4. parser.y part 1: the C code and the declarations

A yacc file has the same three parts as a lex file: C code and declarations, then grammar rules, then plain C, separated by `%%`. This section covers the first part.

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex();
void yyerror(char *s);
extern int yylineno;
extern char *yytext;

int tc = 1, lab = 1;

char *binop(char *op, char *a, char *b) {
    char t[10];
    sprintf(t, "t%d", tc++);
    printf("%s = %s %s %s\n", t, a, op, b);
    return strdup(t);
}

char *text(char *a, char *op, char *b) {
    char s[100];
    sprintf(s, "%s %s %s", a, op, b);
    return strdup(s);
}
%}

%union { char *s; int n; }

%token <s> ID NUM RELOP
%token IF ELSE WHILE FOR
```

```
%type <s> E COND STEP
%type <n> IFH

%nonassoc LOW
%nonassoc ELSE
%left '+' '-'
%left '*' '/'
```

Line	Meaning
`#include <stdlib.h>`	gives `exit`
`int yylex();`	the lexer function from `lexer.l`; the parser calls it for each token
`void yyerror(char *s);`	called by the parser on a syntax error (written at the bottom)
`extern int yylineno; extern char *yytext;`	"these live in the lexer": the current line number and token text, used in the error message
`int tc = 1, lab = 1;`	counters for temporaries (`t1` , `t2` ...) and labels (`L1` , `L2` ...)
`binop(op, a, b)`	makes a new temp, prints `t = a op b` , returns the temp's name. `binop("*", "b", "4")` prints `t1 = b * 4` and returns `"t1"` .
`sprintf(t, "t%d", tc++)`	writes `t` plus the counter into `t` , then adds 1 to the counter
`strdup(t)`	returns a lasting copy; `t` itself disappears when the function ends
`text(a, op, b)`	joins three strings with spaces without printing anything: `text("m", "<", "t1")` gives `"m < t1"`
`%union { char *s; int n; }`	every grammar symbol can carry a value: a string `s` (names, numbers, temps, conditions) or an int `n` (a label number)
`%token <s> ID NUM RELOP`	tokens that carry a string (the lexer set it in `yylval.s`)

Line	Meaning
`%token IF ELSE WHILE FOR`	tokens with no value
`%type <s> E COND STEP`	rules whose value is a string
`%type <n> IFH`	the if-head rule returns a label number
`%nonassoc LOW` / `%nonassoc ELSE`	solve the dangling else (section 6)
`%left '+' '-'` then `%left '*' '/'`	precedence: a later line binds tighter, so `*` and `/` happen before `+` and `-`; `%left` means `a - b - c` is `(a - b) - c`

The last part of the file:

```
void yyerror(char *s) {
    fprintf(stderr, "\nSyntax error at line %d near '%s'\n", yylineno,
yytext);
    exit(1);
}

int main() {
    yyparse();
    return 0;
}
```

`yyparse()` is the parser yacc builds. It keeps calling `yylex()` and runs rule actions until the input ends. On a syntax error it calls `yyerror`, which prints the line and token and stops the program.

5. parser.y part 2: assignments and expressions (questions b and c)

A grammar rule reads `LEFT : piece1 piece2 ... ;` and means "LEFT is made of these pieces in this order". `|` gives another way to make the same LEFT. The code in `{ }` runs the moment the rule matches.

Inside an action:

- `$1`, `$2`, `$3` ... are the values of the pieces, counted left to right.
- `$$` is the value of the LEFT side, handed up to the rule that uses it.
- A rule with no action, like `E : ID`, passes `$1` up automatically.

```
PROG : LIST                { fprintf(stderr, "\nSyntax is valid\n"); }
     ;

LIST : LIST STMT
     | STMT
     ;

STMT : ID '=' E ';'        { printf("%s = %s\n", $1, $3); }
     | '{' LIST '}'
     ...

E    : E '+' E             { $$ = binop("+", $1, $3); }
     | E '-' E             { $$ = binop("-", $1, $3); }
     | E '*' E             { $$ = binop("*", $1, $3); }
     | E '/' E             { $$ = binop("/", $1, $3); }
     | '(' E ')'           { $$ = $2; }
     | ID
     | NUM
     ;
```

Rule	Meaning
`PROG : LIST`	the whole program is a list of statements; once it matches, the syntax was correct
`LIST : LIST STMT | STMT`	one or more statements
`STMT : ID '=' E ';'`	an assignment; `$1` is the name, `$3` is where E's value is; print `name = value`
`STMT : '{' LIST '}'`	a block of statements in braces, usable anywhere one statement is allowed
`E : E '+' E` (and `-`, `*`, `/`)	`binop` prints one TAC line into a new temp and returns that temp
`E : '(' E ')'`	brackets: the value is just the value inside
`E : ID` and `E : NUM`	a name or a number is its own value; no TAC is printed

Trace of `s = x * y + z;`. The parser reads tokens left to right and matches a rule as soon as all its pieces are in:

Step	Rule matched	Value	TAC printed
1	`E : ID` for x	`x`	none
2	`E : ID` for y	`y`	none
3	`E : E '*' E`	`t1`	`t1 = x * y`
4	`E : ID` for z	`z`	none
5	`E : E '+' E`	`t2`	`t2 = t1 + z`
6	`STMT : ID '=' E ';'`	none	`s = t2`

Each rule hands up the *name of the place* holding its result, and the rule above uses that name as an operand. That is syntax-directed translation.

Why `*` goes first. After `x * y` the parser sees `+`. The precedence lines say `*` binds tighter, so it finishes `x * y` before starting the `+`. For `a = b + c * d;` it is the other way round: after `b +` it sees `c *`, and `*` wins, so `c * d` is done first:

```
t1 = c * d
t2 = b + t1
a = t2
```

6. parser.y part 3: while, if, if-else, for

Loops and ifs are built from **labels** (`L1:` marks a place) and **jumps** (`goto L1` always jumps; `if a < b goto L2` jumps only when the test is true). Every statement uses the same three-line opening:

```
if <condition> goto Ltrue
goto Lfalse
Ltrue:
```

Condition. `COND` only builds the text of the test, without printing it:

```
COND : E RELOP E            { $$ = text($1, $2, $3); }
```

For `m < n * 4`, the `E` for `n * 4` prints `t1 = n * 4` and COND returns `"m < t1"`.

Mid-rule actions. A `{ }` placed *between* pieces runs as soon as the pieces before it are read. It counts as a numbered piece and can store a value with `$<n>$ = ...`; later actions read it as `$<n>2`, `$<n>7` and so on. The loops need this because the start label must be printed *before* the condition's TAC.

Label numbers. `lab` is the next free label. A while takes three labels at once (`$<n>$ = lab; lab += 3;`): if it gets 1, it uses L1 (start), L2 (body), L3 (exit).

while

```
| WHILE                 { $<n>$ = lab; lab += 3; printf("L%d:\n", $<n>$); }
  '(' COND ')'          { printf("if %s goto L%d\ngoto L%d\nL%d:\n", $4, $<n>2
+ 1, $<n>2 + 2, $<n>2 + 1); }
  STMT                  { printf("goto L%d\nL%d:\n", $<n>2, $<n>2 + 2); }
```

Pieces: `WHILE` (1), action(2), `'('` (3), `COND` (4), `')'` (5), action(6), `STMT` (7), final action. So `$<n>2` is the start label number and `$4` is the condition text.

```
while (i < 10) { k = 5 * 3; i = i + 1; }

L1:                     <- action 2
if i < 10 goto L2       <- action 6
goto L3
L2:
t1 = 5 * 3              <- the body
k = t1
t2 = i + 1
i = t2
goto L1                 <- final action
L3:
```

if and if-else

```
IFH  : IF '(' COND ')'      { $$ = lab; lab += 2; printf("if %s goto
L%d\ngoto L%d\nL%d:\n", $3, $$, $$ + 1, $$); }

| IFH STMT %prec LOW   { printf("L%d:\n", $1 + 1); }
| IFH STMT ELSE        { $<n>$ = lab++; printf("goto L%d\nL%d:\n", $<n>$, $1
+ 1); }
  STMT                { printf("L%d:\n", $<n>4); }
```

- `IFH` (if-head) handles `if ( cond )` for both forms. It takes two labels: `$$` is the then-label, `$$ + 1` is the false label. Its value is the label number, so `$1` in the rules below is that number.
- **if without else:** after the body, print the false label.
- **if with else:** after the then-body, take a new end label, print `goto end` (skip the else part) and the false label. After the else body, print the end label. Pieces: `IFH` (1) `STMT` (2) `ELSE` (3) action(4) `STMT` (5), so the end label is `$<n>4`.

```
if (x < y) a = 1; else a = 2;

if x < y goto L1
goto L2
L1:
a = 1
goto L3
L2:
a = 2
L3:
```

Dangling else. In `if (a) if (b) x = 1; else x = 2;` the `else` belongs to the nearest `if`. When the parser has `IFH STMT` and sees `else`, it could end the short if or take the else. `%prec LOW` gives the short rule lower priority than `ELSE`, so the parser takes the else. This is why the two `%nonassoc` lines exist, and why yacc reports no conflicts.

for

```
| FOR '(' ID '=' E ';' { printf("%s = %s\n", $3, $5); $<n>$ = lab; lab += 3;
printf("L%d:\n", $<n>$); }
  COND ';' STEP ')'    { printf("if %s goto L%d\ngoto L%d\nL%d:\n", $8, $<n>7
+ 1, $<n>7 + 2, $<n>7 + 1); }
  STMT                 { printf("%s\ngoto L%d\nL%d:\n", $10, $<n>7, $<n>7 +
2); }

STEP : ID '=' ID '+' NUM    { $$ = text($1, "=", text($3, "+", $5)); }
     | ID '=' ID '-' NUM    { $$ = text($1, "=", text($3, "-", $5)); }
```

Pieces: `FOR` (1) `'('` (2) `ID` (3) `'='` (4) `E` (5) `';'` (6) action(7) `COND` (8) `';'` (9) `STEP` (10) `')'` (11) action(12) `STMT` (13) final action.

- Action 7 prints the start, e.g. `i = 0`, then the loop label.

- The step `i = i + 1` is written before the body but must run after it. `STEP` only builds the text `"i = i + 1"`, and the final action prints it after the body. That is why the step must look like `i = i + NUM` or `i = i - NUM`.

```
for (i = 10; i > 0; i = i - 1) s = s + i;

i = 10
L1:
if i > 0 goto L2
goto L3
L2:
t1 = s + i
s = t1
i = i - 1
goto L1
L3:
```

Memorize this table.

Statement	TAC order
while	`L1:`, test, `if ... goto L2`, `goto L3`, `L2:`, body, `goto L1`, `L3:`
for	start, `L1:`, test, `if ... goto L2`, `goto L3`, `L2:`, body, step, `goto L1`, `L3:`
if	test, `if ... goto L1`, `goto L2`, `L1:`, body, `L2:`
if-else	test, `if ... goto L1`, `goto L2`, `L1:`, then-body, `goto L3`, `L2:`, else-body, `L3:`

7. opt.c line by line (question d)

The optimizer reads the TAC, splits every line into words, improves the lines, and prints them back out. It does two passes:

1. **simplify()** on every line: constant folding, algebraic simplification, strength reduction.
2. **motion()** on every loop: code motion.

How a line is stored

```
char w[300][6][30];
int n[300], pos[300], N = 0;
```

- `w[i][k]` is word `k` of line `i`. The line `t1 = n * 4` is stored as `w[i][0]="t1"`, `w[i][1]="="`, `w[i][2]="n"`, `w[i][3]="*"`, `w[i][4]="4"`.
- `n[i]` is how many words line `i` has: 5 for `t1 = n * 4`, 3 for `r = t2`, 2 for `goto L1`, 1 for `L1:`.
- `pos[i]` is -1 normally. If line `i` is moved out of a loop, `pos[i]` holds the line number of that loop's label.
- `N` counts the lines.

Small helpers

```
int isnum(char *s) {
    if (!*s) return 0;
    while (*s) if (!isdigit(*s++)) return 0;
    return 1;
}
int istemp(char *s) { return s[0] == 't' && isdigit(s[1]); }
int is(int i, int k, char *s) { return strcmp(w[i][k], s) == 0; }
void copy(int i, char *v) { char t[30]; strcpy(t, v); strcpy(w[i][2], t);
n[i] = 3; }
```

Helper	Meaning
`isnum(s)`	1 if `s` is all digits (`"16"` yes, `"x"` no)
`istemp(s)`	1 if `s` is a temp: `t` followed by a digit (`t3` yes, `t` or `x` no)
`is(i, k, s)`	1 if word `k` of line `i` equals `s`
`copy(i, v)`	turns line `i` into `dest = v` by putting `v` in word 2 and cutting it to 3 words. It copies through `t` first because `v` may itself be word 2 or word 4 of the same line.

simplify(i): three techniques

It only looks at 5-word lines `dest = a op b`; `a`, `op` and `b` are short names for words 2, 3 and 4.

```
if (isnum(a) && isnum(b)) {
    int x = atoi(a), y = atoi(b), r;
    if (!strcmp(op, "+")) r = x + y;
    else if (!strcmp(op, "-")) r = x - y;
    else if (!strcmp(op, "*")) r = x * y;
    else if (y != 0) r = x / y;
    else return;
    char s[30];
    sprintf(s, "%d", r);
    copy(i, s);
}
```

Constant folding: when both operands are numbers, work out the answer now. `atoi` turns text into a number, `sprintf` turns the answer back into text, and `copy` makes the line `dest = answer`. Division by 0 is left alone. `!strcmp(op, "+")` means "op is +" (`strcmp` returns 0 when the strings are equal).

```
else if ((!strcmp(op, "+") || !strcmp(op, "-")) && !strcmp(b, "0")) copy(i,
a);
else if (!strcmp(op, "+") && !strcmp(a, "0"))                       copy(i,
b);
else if ((!strcmp(op, "*") || !strcmp(op, "/")) && !strcmp(b, "1")) copy(i,
a);
else if (!strcmp(op, "*") && !strcmp(a, "1"))                       copy(i,
b);
else if (!strcmp(op, "*") && (!strcmp(a, "0") || !strcmp(b, "0")))  copy(i,
"0");
```

Algebraic simplification: identities. `x + 0`, `x - 0` and `0 + x` become `x`; `x * 1`, `x / 1` and `1 * x` become `x`; `x * 0` and `0 * x` become `0`.

```
else if (!strcmp(op, "*") && !strcmp(b, "2")) { strcpy(op, "+");  strcpy(b,
a); }
else if (!strcmp(op, "*") && !strcmp(b, "4")) { strcpy(op, "<<"); strcpy(b,
"2"); }
else if (!strcmp(op, "*") && !strcmp(b, "8")) { strcpy(op, "<<"); strcpy(b,
"3"); }
```

Strength reduction: replace multiplication with something cheaper. `x * 2` becomes `x + x`; `x * 4` becomes `x << 2` (shift left 2 = multiply by 4); `x * 8` becomes `x << 3`.

Technique	Before	After
Constant folding	`t2 = 16 * 5`	`t2 = 80`
Algebraic simplification	`t1 = a + 0`	`t1 = a`
Algebraic simplification	`t2 = b * 0`	`t2 = 0`
Strength reduction	`t1 = n * 4`	`t1 = n << 2`
Strength reduction	`t1 = a * 2`	`t1 = a + a`

motion(s): code motion

A line inside a loop is **loop-invariant** when it gives the same value on every pass. It can be computed once, before the loop. `motion` is called with `s`, the line number of a label.

```
char name[30];
strcpy(name, w[s][0]);
name[strlen(name) - 1] = '\0';

int e = -1;
for (int j = s + 1; j < N; j++)
    if (n[j] == 2 && is(j, 0, "goto") && is(j, 1, name)) e = j;
if (e < 0) return;
```

Copy the label (`L1:`) and cut off the `:` to get `L1`. Then look for a later `goto L1`. Only a loop jumps *back* to its label, so if there is none, this label is not a loop and we stop. The loop body is everything between line `s` and line `e`.

```
for (int k = s + 1; k < e; k++) {
    if (pos[k] >= 0 || n[k] < 3 || !is(k, 1, "=") || !istemp(w[k][0]))
continue;
    if (n[k] == 5 && is(k, 3, "/")) continue;
    if (assigned(w[k][2], s, e)) continue;
    if (n[k] == 5 && assigned(w[k][4], s, e)) continue;
    pos[k] = s;
}
```

For each line in the loop, skip it (`continue`) unless all of these hold:

1. it has not been moved already, and it is an assignment to a **temp**. Assignments to real variables like `r = t2` stay, so a loop that never runs leaves them unchanged.
2. it is not a division; moving it could divide by zero when the loop never runs.
3. its first operand is not assigned anywhere in the loop.
4. its second operand (if it has one) is not assigned anywhere in the loop.

If it passes, `pos[k] = s` marks it "move to just before the label".

```
int assigned(char *v, int s, int e) {
    for (int j = s + 1; j < e; j++)
        if ((pos[j] < 0 || pos[j] > s) && n[j] >= 3 && is(j, 1, "=") && is(j,
0, v)) return 1;
    return 0;
}
```

`assigned` answers "is `v` given a value inside this loop?" It checks every line between `s` and `e` of the form `v = ...`. A line already moved out of *this* loop (`pos[j] == s`) does not count, so a chain like `t3 = x * y` then `t4 = t3 + z` can both move. A line moved only out of an inner loop (`pos[j] > s`) is still inside this loop, so it does count.

main()

```
while (fgets(buf, 200, stdin)) {
    char *p = strtok(buf, " \t\r\n");
    if (!p) continue;
    n[N] = 0;
    while (p) { strcpy(w[N][n[N]++], p); p = strtok(NULL, " \t\r\n"); }
    pos[N] = -1;
    N++;
}
```

Read each line. `strtok` splits it at spaces: the first call gives the first word, each later call with `NULL` gives the next. Empty lines are skipped.

```
for (int i = 0; i < N; i++)
    simplify(i);

for (int i = N - 1; i >= 0; i--)
    if (n[i] == 1) motion(i);
```

Simplify every line, then try code motion at every label (a 1-word line). Going from the bottom up handles an inner loop before the loop around it.

```
for (int i = 0; i < N; i++) {
    for (int k = 0; k < N; k++)
        if (pos[k] == i) print(k);
    if (pos[i] < 0) print(i);
}
```

Print the result. Before each line `i`, first print every line marked to move there; then print line `i` itself unless it was moved. `print(i)` writes the words of line `i` with spaces between them.

The exam sample, traced

Line in the loop	Pass 1	Moved out?	Why
`t1 = n * 4`	`t1 = n << 2`	yes	temp; `n` is never assigned in the loop
`t2 = 16 * 5`	`t2 = 80`	yes	temp; `80` is a number
`r = t2`		no	`r` is a real variable
`t3 = x * y`		yes	`x` and `y` never assigned in the loop
`t4 = t3 + z`		yes	`t3` was moved out; `z` never assigned
`s = t4`		no	real variable
`t5 = m + s`		no	`s` is assigned in the loop
`t = t5`		no	real variable

The `if` and `goto` lines never move: they are not assignments.

In `opt.c` each technique is marked with a one-line comment (`// Constant folding`, `// Algebraic simplification`, `// Strength reduction`, `// Code motion (loop invariant)`), so you can point to it in the exam.

Add-on: dead code elimination (commented out)

Dead code is an assignment to a temp whose value is never used. Removing it changes nothing in the result. The add-on sits in `opt.c` inside `/* ... */`:

```
int used(char *v) {
    for (int j = 0; j < N; j++)
        for (int k = 1; k < n[j]; k++)
            if (is(j, k, v)) return 1;
    return 0;
}

void deadcode() {
    for (int i = N - 1; i >= 0; i--)
        if (n[i] >= 3 && is(i, 1, "=") && istemp(w[i][0]) && !used(w[i][0]))
            n[i] = 0;
}
```

- `used(v)` checks every word of every line except word 0, because word 0 of an assignment is the variable being *set*, not used.
- `deadcode()` looks for lines `tN = ...` where `tN` is never used, and deletes them by setting their word count to 0. A line with 0 words is skipped by printing and by code motion.
- It goes from the bottom up: removing an unused `t2 = t1 + 1` can make `t1` unused too, and that line is reached next.

To switch it on: delete the `/*` and `*/` lines around it, and remove the `//` from `// deadcode();` in `main`. It runs after `simplify` and before code motion.

The parser never creates an unused temp, so on normal programs nothing is removed. To demo it, feed TAC directly: `./opt < tests/deadcode_tac.txt`.

```
Input            Output
t1 = 2 * 3       t1 = 6
t2 = a + 0       t3 = b << 2
t3 = b * 4       x = t1 + t3
t4 = c - d
x = t1 + t3
```

`t2` and `t4` are never used, so they are removed. The other lines are also folded and strength-reduced.

8. gen.c line by line (question e)

The code generator reads the optimized TAC one line at a time, works out which of five shapes the line has, and prints the matching assembly. It uses one register, `R0`. `MOV dest, src` copies `src` into `dest`.

```
char *jump(char *op) {
    if (!strcmp(op, "<"))  return "JL";
    if (!strcmp(op, ">"))  return "JG";
    if (!strcmp(op, "<=")) return "JLE";
    if (!strcmp(op, ">=")) return "JGE";
    if (!strcmp(op, "==")) return "JE";
    return "JNE";
}

char *instr(char *op) {
    if (!strcmp(op, "+")) return "ADD";
    if (!strcmp(op, "-")) return "SUB";
    if (!strcmp(op, "*")) return "MUL";
    if (!strcmp(op, "/")) return "DIV";
    return "SHL";
}
```

`jump` picks the conditional jump for a comparison; `instr` picks the instruction for an arithmetic operator (`<<` gives `SHL`, shift left).

Comparison	Jump	Meaning
`<`	`JL`	jump if less
`>`	`JG`	jump if greater
`<=`	`JLE`	jump if less or equal
`>=`	`JGE`	jump if greater or equal
`==`	`JE`	jump if equal
`!=`	`JNE`	jump if not equal

```
while (fgets(l, 200, stdin)) {
    if (sscanf(l, "if %s %s %s goto %s", a, op, b, lab) == 4) {
        printf("CMP %s, %s\n", a, b);
        printf("%s %s\n", jump(op), lab);
    }
    else if (sscanf(l, "goto %s", lab) == 1)
        printf("JMP %s\n", lab);
    else if (strchr(l, ':'))
        printf("%s", l);
    else if (sscanf(l, "%s = %s %s %s", d, a, op, b) == 4) {
        printf("MOV R0, %s\n", a);
        printf("%s R0, %s\n", instr(op), b);
        printf("MOV %s, R0\n", d);
    }
    else if (sscanf(l, "%s = %s", d, a) == 2) {
        printf("MOV R0, %s\n", a);
        printf("MOV %s, R0\n", d);
    }
}
```

`sscanf(line, pattern, ...)` reads a line using a pattern and returns how many `%s` parts it filled. Each `if` tries one shape; the first that fits wins.

TAC shape	Test	Assembly
`if a < b goto L2`	`sscanf` fills 4 parts	`CMP a, b` then `JL L2`
`goto L1`	fills 1 part	`JMP L1`
`L1:`	the line has a `:`	`L1:` unchanged
`t1 = a + b`	fills 4 parts	`MOV R0, a`, `ADD R0, b`, `MOV t1, R0`
`r = t2`	fills 2 parts	`MOV R0, t2`, `MOV r, R0`

One order matters: the 4-part assignment is tried before the 2-part one, because `t1 = a + b` would also match `%s = %s` (it would read only `t1` and `a`).

Example. `x = a * b + c;` has TAC `t1 = a * b`, `t2 = t1 + c`, `x = t2`, which becomes:

```
MOV R0, a
MUL R0, b
MOV t1, R0
MOV R0, t1
ADD R0, c
MOV t2, R0
MOV R0, t2
MOV x, R0
```

If the exam asks for assembly of "one or two given statements", you can also do it by hand with this table.

9. run.sh and the full output for the exam sample (question f)

`run.sh` is the integration: it builds the four programs and runs them in a chain, each one reading the file the previous one wrote.

```
#!/bin/sh
IN=${1:-input.txt}

bison -d parser.y || exit 1
flex lexer.l || exit 1
gcc parser.tab.c lex.yy.c -o icg || exit 1
gcc opt.c -o opt || exit 1
gcc gen.c -o gen || exit 1

echo "===== INPUT ====="
cat $IN
echo
echo "===== (a) TOKENS + (b) SYNTAX CHECK ====="
./icg < $IN > tac.txt || exit 1
echo
echo "===== (c) THREE ADDRESS CODE ====="
cat tac.txt
echo
echo "===== (d) OPTIMIZED CODE ====="
./opt < tac.txt > opt.txt
cat opt.txt
echo
echo "===== (e) TARGET CODE ====="
./gen < opt.txt
```

Line	Meaning
`IN=${1:-input.txt}`	the input file is the first argument, or `input.txt` if none is given
`|| exit 1`	if this step fails (a build error, or a syntax error), stop the script
`./icg < $IN > tac.txt`	run the lexer and parser: tokens and the syntax message go to the screen (`stderr`), TAC goes into `tac.txt`
`./opt < tac.txt > opt.txt`	optimize `tac.txt` into `opt.txt`
`./gen < opt.txt`	print the assembly
`cat`	show a file; `echo` prints a heading

Output for the exam sample

```
while (m < n * 4)
{
    r = 16 * 5;
    s = x * y + z;
    t = m + s;
}
```

(a) and (b): one line per token (`while keyword` , `( special symbol` , `m identifier` , `< relational operator` …), then `Syntax is valid` .

(c) Three-address code

```
L1:
t1 = n * 4
if m < t1 goto L2
goto L3
L2:
t2 = 16 * 5
r = t2
t3 = x * y
t4 = t3 + z
s = t4
t5 = m + s
t = t5
goto L1
L3:
```

(d) Optimized code. `n * 4` became `n << 2` (strength reduction), `16 * 5` became `80` (constant folding), and four invariant temp lines moved above `L1` (code motion).

```
t1 = n << 2
t2 = 80
t3 = x * y
t4 = t3 + z
L1:
if m < t1 goto L2
goto L3
L2:
r = t2
s = t4
t5 = m + s
t = t5
goto L1
L3:
```

(e) Target code

```
MOV R0, n
SHL R0, 2
MOV t1, R0
MOV R0, 80
MOV t2, R0
MOV R0, x
MUL R0, y
```

```
MOV t3, R0
MOV R0, t3
ADD R0, z
MOV t4, R0
L1:
CMP m, t1
JL L2
JMP L3
L2:
MOV R0, t2
MOV r, R0
MOV R0, t4
MOV s, R0
MOV R0, m
ADD R0, s
MOV t5, R0
MOV R0, t5
MOV t, R0
JMP L1
L3:
```

Two more to check against

`for (i = 0; i < 10; i = i + 1) { x = a * 2; y = i * 8; s = s + i; }` optimizes to the code below. `a * 2` became `a + a` and moved out (it never changes). `i * 8` became `i << 3` but stayed, because `i` changes every pass.

```
i = 0
t1 = a + a
L1:
if i < 10 goto L2
goto L3
L2:
x = t1
t2 = i << 3
y = t2
t3 = s + i
s = t3
i = i + 1
goto L1
L3:
```

`if (a == b) c = a * 1; else c = b * 0;` optimizes to the code below. `a * 1` became `a` and `b * 0` became `0`; there is no loop, so nothing moves.

```
if a == b goto L1
goto L2
L1:
t1 = a
c = t1
goto L3
L2:
t2 = 0
c = t2
L3:
```

10. Practice: work these out on paper, then check

Cover the answers, write your own, then compare. Every answer is the program's real output; you can also run any of them with `sh run.sh` on a file holding the input.

1. TAC for `a = b + c * d;`
2. TAC, then optimized TAC, for `x = (a + b) * 2;`
3. TAC for `if (x < y) a = 1; else a = 2;`
4. TAC and optimized TAC for `while (i < 10) { k = 5 * 3; i = i + 1; }`. Which line moves out, and why does the rest stay?
5. TAC for `for (i = 10; i > 0; i = i - 1) s = s + i;`. Does anything move out of the loop?
6. Assembly for `x = a * b + c;`
7. Assembly for the TAC line `if i < 10 goto L2`, then for `if a != b goto L5`.

Answers

1. `*` binds tighter, so it goes first:

```
t1 = c * d
t2 = b + t1
a = t2
```

2. The brackets force the addition first. Optimizing turns `* 2` into an addition:

```
TAC              Optimized
t1 = a + b       t1 = a + b
t2 = t1 * 2      t2 = t1 + t1
x = t2           x = t2
```

3. The if-else pattern:

```
if x < y goto L1
goto L2
L1:
a = 1
goto L3
L2:
a = 2
L3:
```

4.

```
TAC                  Optimized
L1:                  t1 = 15
if i < 10 goto L2    L1:
goto L3              if i < 10 goto L2
L2:                  goto L3
t1 = 5 * 3           L2:
k = t1               k = t1
t2 = i + 1           t2 = i + 1
i = t2               i = t2
goto L1              goto L1
L3:                  L3:
```

`5 * 3` folds to `15`, and `t1 = 15` moves out: it is a temp and 15 never changes. `k = t1` stays because `k` is a real variable. `t2 = i + 1` stays because `i` is assigned in the loop.

5.

```
i = 10
L1:
if i > 0 goto L2
goto L3
L2:
t1 = s + i
s = t1
i = i - 1
goto L1
L3:
```

Nothing moves: `s` and `i` both change inside the loop, so `t1 = s + i` is not invariant.

6. TAC `t1 = a * b`, `t2 = t1 + c`, `x = t2`, so:

```
MOV R0, a
MUL R0, b
MOV t1, R0
MOV R0, t1
ADD R0, c
MOV t2, R0
MOV R0, t2
MOV x, R0
```

7. `CMP i, 10` then `JL L2`. And `CMP a, b` then `JNE L5`.

If you got 1, 2 and 6, you can do any assignment question. If you got 3, 4 and 5, you know the control-statement and code-motion patterns, which is where most of the marks are.

11. Writing it from memory, common errors, limits

Order to write it in the exam. Build and run after each step, so a mistake shows up while it is small. Each step is a module you can show for marks.

1. `lexer.l`: keywords, RELOP, ID, NUM, operators, symbols, whitespace, `yywrap`. About 25 lines.
2. `parser.y` with only `PROG`, `LIST`, the assignment and the `E` rules, plus `binop`, `yyerror` and `main`. Run it on `a = b + c * d;`. This already answers (b) and (c) for assignments.
3. Add `COND`, then `while`, then `IFH` with if and if-else, then `for` with `STEP`. Test each.
4. `opt.c`: `simplify` first, then `motion`.

5. `gen.c`.
6. `run.sh`.

Things to remember

- Precedence: `%left '+' '-'` goes *above* `%left '*' '/'`.
- Piece numbers: a mid-rule action counts as a piece. In `while` the start label is `$<n>2` and the condition is `$4`. In `for` the label is `$<n>7`, the condition `$8`, the step `$10`.
- The three-line opening of every condition: `if c goto Ltrue`, `goto Lfalse`, `Ltrue:`.
- Code motion moves only **temps**, never divisions, and only when neither operand is assigned inside the loop.

Common errors

Error	Fix
`parser.tab.h: No such file`	run `bison -d parser.y` before `flex` and `gcc`
`undefined reference to yywrap`	keep `int yywrap() { return 1; }` at the end of `lexer.l`
`Syntax error at line N` on input that looks right	check for a missing `;` (every assignment needs one), an unclosed `{`, or a `for` step that is not `i = i + NUM` / `i = i - NUM`
`invalid character` in the token list	the input has a character the lexer does not know, such as `%` or `&`
`conflicts` warning from bison	a precedence line or the `%prec LOW` was left out

Limits of this version

- Statements: assignment, `if`, `if-else`, `while`, `for`, and `{ }` blocks. Every statement ends with `;`.
- Expressions: `+ - * /` with brackets, on variables and whole numbers. No unary minus (write `0 - x`).
- Conditions: one comparison only (`< > <= >= == !=`), no `&&`, `||` or `!`.
- The `for` step must be `i = i + NUM` or `i = i - NUM`.

- The optimizer folds one line at a time. `r = 2 * 3 + 4;` becomes `t1 = 6` then `t2 = t1 + 4`; it does not carry the 6 into the next line.
- Do not name variables like temps or labels (`t1` , `L1`).